

AFRILANG Stdlib

std/c/cryptsha1

Bibliothèque AFRILANG `std/c/cryptsha1` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : sha1Len, sha1Up

```
`import "std/c/cryptsha1" . `use cryptsha1`
```

- `sha1Len(s text) ? number`
- `sha1Upper(s text) ? text`
- `sha1Lower(s text) ? text`
- `sha1Trim(s text) ? text`
- `sha1Split(s text, delim text) ? list text`
- `sha1Join(parts list text, delim text) ? text`
- `sha1Replace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/cryptsha1` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : sha1Len, sha1Up

```
`import "std/c/cryptsha1" . `use cryptsha1`  
- `sha1Len(s text) ? number`  
- `sha1Upper(s text) ? text`  
- `sha1Lower(s text) ? text`  
- `sha1Trim(s text) ? text`  
- `sha1Split(s text, delim text) ? list text`  
- `sha1Join(parts list text, delim text) ? text`  
- `sha1Replace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptsha1"  
use cryptsha1
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? sha1Len(s text) ? number  
? sha1Upper(s text) ? text  
? sha1Lower(s text) ? text  
? sha1Trim(s text) ? text  
? sha1Split(s text, delim text) ? list  
? sha1Join(parts list text, delim text) ? text  
? sha1Replace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Exemples d'utilisation

```
import "std/c/cryptsha1"  
use cryptsha1  
  
create result = sha1Len(/* args */)   
say result  
  
create result = sha1Upper(/* args */)   
say result  
  
create result = sha1Lower(/* args */)   
say result  
  
create result = sha1Trim(/* args */)   
say result  
  
create result = sha1Split(/* args */)   
say result  
  
create result = sha1Join(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/cryptsha1"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module cryptsha1
function sha1Len(s text) returns number
end
function sha1Upper(s text) returns text
end
function sha1Lower(s text) returns text
end
function sha1Trim(s text) returns text
end
function sha1Split(s text, delim text) returns list text
end
function sha1Join(parts list text, delim text) returns text
end
function sha1Replace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/cryptsha1` (tier complex, category complex). Exports 8 function(s): sha1Len, sha1Upper, sha1Lower

```
`import "std/c/cryptsha1" . `use cryptsha1`  
- `sha1Len(s text) ? number`  
- `sha1Upper(s text) ? text`  
- `sha1Lower(s text) ? text`  
- `sha1Trim(s text) ? text`  
- `sha1Split(s text, delim text) ? list text`  
- `sha1Join(parts list text, delim text) ? text`  
- `sha1Replace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptsha1"  
use cryptsha1
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? sha1Len(s text) ? number  
? sha1Upper(s text) ? text  
? sha1Lower(s text) ? text  
? sha1Trim(s text) ? text  
? sha1Split(s text, delim text) ? list  
? sha1Join(parts list text, delim text) ? text  
? sha1Replace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Usage examples

```
import "std/c/cryptsha1"  
use cryptsha1  
  
create result = sha1Len(/* args */)   
say result  
  
create result = sha1Upper(/* args */)   
say result  
  
create result = sha1Lower(/* args */)   
say result  
  
create result = sha1Trim(/* args */)   
say result  
  
create result = sha1Split(/* args */)   
say result  
  
create result = sha1Join(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/cryptsha1"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module cryptsha1
function sha1Len(s text) returns number
end
function sha1Upper(s text) returns text
end
function sha1Lower(s text) returns text
end
function sha1Trim(s text) returns text
end
function sha1Split(s text, delim text) returns list text
end
function sha1Join(parts list text, delim text) returns text
end
function sha1Replace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```